LTLTRA001 | Tracey Letlape | CSC3042F

Multi-Layer Perceptron (MLP) vs Convolutional Neural Network (CNN)

PART A: Complexity Ladder

- A4.1 Implementation: Code for both architectures and training loop
- A4.2 Results table: Test Accuracy vs Dataset combinations
- A4.3 Training Curves: Loss and accuracy plots on a single figure
- A4.4 Written Analysis
- A4.5 Reflection

1. Imports and Seeds

In [1]:
```
# ChatGPT was used for debuggginf and conceptual clarification

# Necessary imports
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, random_split, Subset

import numpy as np
import pandas as pd
import random
import matplotlib.pyplot as plt
import time
```

In [2]:
```
SEED = 42

random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
torch.cuda.manual_seed(SEED)
torch.cuda.manual_seed_all(SEED)

torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
```

In [3]:
```
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f'Using device: {DEVICE}')
print(f'PyTorch version: {torch.__version__}')
```

```
Using device: cuda
PyTorch version: 2.10.0+cu128
```

2. Dataset Loaders

- To keep the MNIST and the CIFAR datasets the same, resize all images to 32x32

In [4]:
```
BATCH_SIZE = 128

# Define MNIST normalisation parameters
MNIST_MEAN = (0.1307,)
MNIST_STD = (0.30811,)

# Define CIFAR Normalisation parameters
CIFAR_MEAN = (0.4914, 0.4822, 0.4465)
CIFAR_STD  = (0.2470, 0.2435, 0.2616)

# Define a MNIST transfom
_mnist_transform = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
    transforms.Normalize(MNIST_MEAN, MNIST_STD),
])

_cifar_transform = transforms.Compose([
    transforms.Resize((32, 32)),
    transforms.ToTensor(),
    transforms.Normalize(CIFAR_MEAN, CIFAR_STD)
])
```

In [5]:
```
# Define a reusable helper method to split a dataset
def _split_dataset(dataset: datasets.MNIST | datasets.CIFAR10, train_ratio: floa
    # Compute split sizes
    total_size = len(dataset)

    train_size = int(train_ratio * total_size)
    val_size = total_size - train_size

    # Reproducible split
    generator = torch.Generator().manual_seed(seed)
    train_dataset, val_dataset = random_split(
        dataset,
        [train_size, val_size],
        generator=generator
    )

    return train_dataset, val_dataset
```

In [6]:
```
# Get the MNIST datasets
_mnist_full_train_dataset = datasets.MNIST(
    root="./kaggle/working",
    train=True,
    download=True,
    transform=_mnist_transform
)

MNIST_TEST_DATASET = datasets.MNIST(
    root="./kaggle/working",
    train=False,
    download=True,
    transform=_mnist_transform
)

# Split the training dataset into train and val datasets
```

```
MNIST_TRAIN_DATASET, MNIST_VAL_DATASET = _split_dataset(_mnist_full_train_datase

_cifar_full_train_dataset = datasets.CIFAR10(
    root="./kaggle/working",
    train=True,
    download=True,
    transform=_cifar_transform
)

CIFAR_TEST_DATASET = datasets.CIFAR10(
    root="./kaggle/working",
    train=False,
    download=True,
    transform=_cifar_transform,
)

# Split the training dataset into train and val datasets
CIFAR_TRAIN_DATASET, CIFAR_VAL_DATASET = _split_dataset(_cifar_full_train_datase
print()
print(f"MNIST Train size: {len(MNIST_TRAIN_DATASET)}")
print(f"MNIST validation size: {len(MNIST_VAL_DATASET)}")
print(f"MNIST Test size: {len(MNIST_TEST_DATASET)}")
print()

print(f"CIFAR10 Train size: {len(CIFAR_TRAIN_DATASET)}")
print(f"CIFAR10 validation size: {len(CIFAR_VAL_DATASET)}")
print(f"CIFAR10 Test size: {len(CIFAR_TEST_DATASET)}")
```

```
100%|██████████| 9.91M/9.91M [00:00<00:00, 39.5MB/s]
100%|██████████| 28.9k/28.9k [00:00<00:00, 1.21MB/s]
100%|██████████| 1.65M/1.65M [00:00<00:00, 10.8MB/s]
100%|██████████| 4.54k/4.54k [00:00<00:00, 17.9MB/s]
100%|██████████| 170M/170M [00:14<00:00, 11.9MB/s]
MNIST Train size: 48000
MNIST validation size: 12000
MNIST Test size: 10000

CIFAR10 Train size: 40000
CIFAR10 validation size: 10000
CIFAR10 Test size: 10000
```

In [7]:
```
"""Define the data lodaers for the datasets."""
MNIST_TRAIN_LOADER = DataLoader(MNIST_TRAIN_DATASET, batch_size=BATCH_SIZE, shuf
MNIST_VAL_LOADER = DataLoader(MNIST_VAL_DATASET, batch_size=BATCH_SIZE, shuffle=
MNIST_TEST_LOADER = DataLoader(MNIST_TEST_DATASET, batch_size=BATCH_SIZE, shuffl

CIFAR_TRAIN_LOADER = DataLoader(CIFAR_TRAIN_DATASET, batch_size=BATCH_SIZE, shuf
CIFAR_VAL_LOADER = DataLoader(CIFAR_VAL_DATASET, batch_size=BATCH_SIZE, shuffle=
CIFAR_TEST_LOADER = DataLoader(CIFAR_TEST_DATASET, batch_size=BATCH_SIZE, shuffl
```

3. Multi-Layer Perceptron

- Input, 3 Hidden Layers, Output
- Architecture: [input_dim -> 512 -> 256 -> 128 -> num_classes]
- Uses the ReLU activation function for all layers except the output layer which uses sigmoid.

In [8]:
```python
# Define a fixed MLP
class MLP(nn.Module):
    """
        Feedforward network implementing MLP.
        Architecture: [input_dim -> *hidden_dims -> output_dim]
    """
    def __init__(self, input_dim, num_classes: int = 10, dropout: float = 0.3):
        super().__init__()
        self.h1 = nn.Linear(input_dim, 512)
        self.h2 = nn.Linear(512, 256)
        self.h3 = nn.Linear(256, 128)
        self.out = nn.Linear(128, num_classes)
        self.dropout = nn.Dropout(dropout)      # Define once, reuse

    def forward(self, x):
        x = torch.flatten(x, start_dim=1)

        # First hidden layer
        self.a1 = F.relu(self.h1(x))
        self.a1 = self.dropout(self.a1)

        # Second hidden layer
        self.a2 = F.relu(self.h2(self.a1))
        self.a2 = self.dropout(self.a2)

        # Third hidden layer
        self.a3 = F.relu(self.h3(self.a2))
        self.a3 = self.dropout(self.a3)

        # Output
        return self.out(self.a3)
```

4. Convolutional Neural Network (CNNs)

- Defined a reusable ConvBlock class and the CNN model class
- Input, 3 ConvBlocks, 1 FC hidden layer
- Uses the ReLU activation function for all layers except the output layer which uses sigmoid.

In [9]:
```python
class ConvBlock(nn.Module):
    """
    Conv2d -> Batchnorm -> ReLU
    """

    def __init__(self, in_channels, out_channels):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv2d(in_channels, out_channels, kernel_size=3,
                      padding=1, bias=False
            ),
            nn.BatchNorm2d(out_channels),
            nn.ReLU(inplace=True)
        )

    def forward(self, x):
        return self.block(x)
```

In [10]:
```
class CNN(nn.Module):
    def __init__(self, input_channels, num_classes=10, dropout=0.5):
        super().__init__()

        # ── Stage 1: extract low-level features ──────────────────────
        self.stage1 = nn.Sequential(
            ConvBlock(input_channels,  32),
            ConvBlock(32, 32),
            nn.MaxPool2d(2, 2),             # HxW -> H/2 x W/2
        )

        # ── Stage 2: mid-level features ──────────────────────────────
        self.stage2 = nn.Sequential(
            ConvBlock(32, 64),
            ConvBlock(64, 64),
            nn.MaxPool2d(2, 2),             # H/2 x W/2 -> H/4 x W/4
        )

        # ── Stage 3: higher-level features ───────────────────────────
        self.stage3 = nn.Sequential(
            ConvBlock(64, 128),
            ConvBlock(128, 128),
        )

        # ── Classifier head ──────────────────────────────────────────
        self.head = nn.Sequential(
            nn.AdaptiveAvgPool2d((1, 1)),       # 128xH'xW' -> 128x1x1
            nn.Flatten(),                       #  -> 128
            nn.Linear(128, 256),
            nn.ReLU(),
            nn.Dropout(dropout),
            nn.Linear(256, num_classes),
        )

    def forward(self, x):
        x = self.stage1(x)
        x = self.stage2(x)
        x = self.stage3(x)
        x = self.head(x)
        return x
```

5. Training and Evaluation FUnctions

In [11]:
```
def train_model(model: MLP | CNN, loader: DataLoader, criterion: nn.Module, opti
    # Move model to device
    model.to(device)

    # Set model to training mode
    model.train()

    total_loss, total_correct, total_samples = 0.0, 0, 0

    for images, labels in loader:
        images, labels = images.to(device), labels.to(device)

        # Reset gradients to zero
        optimiser.zero_grad()
```

```
        # Forward pass & loss computation
        logits = model(images)
        loss = criterion(logits, labels)

        # Backward pass & parameter update
        loss.backward()
        optimiser.step()

        # Update metrics
        batch_size = images.size(0)

        total_loss += loss.item() * batch_size
        total_correct += (logits.argmax(1) == labels).sum().item()
        total_samples += batch_size

    # Compute avarage metrics
    avg_loss = total_loss / total_samples
    avg_acc = total_correct / total_samples

    return avg_loss, avg_acc
```

In [12]:
```
def evaluate_model(model: MLP | CNN, loader: DataLoader, criterion: nn.Module, d
    # Move model to device
    model.to(device)

    # Set the model to evaluation mode
    model.eval()

    total_loss, total_correct, total_samples = 0.0, 0, 0

    with torch.no_grad():
        for images, labels in loader:
            images, labels = images.to(device), labels.to(device)

            # Forward pass
            logits = model(images)
            loss = criterion(logits, labels)

            # Update metrics
            batch_size = images.size(0)

            total_loss += loss.item() * batch_size
            total_correct += (logits.argmax(1) == labels).sum().item()
            total_samples += batch_size

    # Compute avarage metrics
    avg_loss = total_loss / total_samples
    avg_acc = total_correct / total_samples

    return avg_loss, avg_acc
```

6. Define the training loop with same settings for all models

In [13]:
```
def training_loop(model: MLP | CNN, train_loader: DataLoader, val_loader: DataLo
    # Define the patience limit to stop when there is no improvement
    PATIENCE = 5

    # Define the metrics to track performance
```

```
    best_val_loss = float("inf")
    epochs_no_improve = 0

    # Move model to device
    model = model.to(device)

    # Use CrossEntropyLoss as the loss function
    criterion = nn.CrossEntropyLoss()

    # Use AdamW with weight_decay as 1e-4 as the optimiser
    optimiser = optim.AdamW(model.parameters(), lr=lr, weight_decay=1e-4)

    # Define a scheduler
    scheduler = optim.lr_scheduler.CosineAnnealingLR(optimiser, T_max=epochs)

    # Store metrics
    history = {
        'epoch': [],
        'train_loss': [],
        'val_loss': [],
        'train_acc': [],
        'val_acc': [],
        'lr': [],
        'epochs': []
    }

    # Keep track of the time
    start_time = time.time()

    # Run through epochs
    for epoch in range(1, epochs+1):
        tr_loss, tr_acc = train_model(model, train_loader, criterion, optimiser,
        va_loss, va_acc = evaluate_model(model, val_loader, criterion, device)

        current_lr = scheduler.get_last_lr()[0]

        history['epoch'].append(epoch)
        history['train_loss'].append(tr_loss)
        history['val_loss'].append(va_loss)
        history['train_acc'].append(tr_acc)
        history['val_acc'].append(va_acc)
        history['lr'].append(current_lr)
        history['epochs'].append(epoch)

        scheduler.step()

        if va_loss < best_val_loss:
            best_val_loss = va_loss
            epochs_no_improve = 0   # Reset state
        else:
            epochs_no_improve += 1
            if epochs_no_improve >= PATIENCE:
                print(f"Early stopping at epoch {epoch}")
                break

    total_time = time.time() - start_time

    test_loss, test_acc = evaluate_model(
        model, test_loader, criterion, device
    )
```

```
    return model, history, test_loss, test_acc, total_time
```

Run The Training Loop for 4 different models using the same settings

In [14]:
```
EPOCHS = 10
LR = 0.001
DROPOUT = 0.3
```

MNIST MLP

In [15]:
```
# Train the MLP model on MNIST data using default parameters
mnist_mlp = MLP(input_dim=1*32*32, dropout=DROPOUT)

_, history_mnist_mlp, tl_mnist_mlp, ta_mnist_mlp, time_mnist_mlp = training_loop
    mnist_mlp,
    MNIST_TRAIN_LOADER,
    MNIST_VAL_LOADER,
    MNIST_TEST_LOADER,
    lr=LR,
    epochs=EPOCHS,
    device=DEVICE
)
```

MNIST CNN

In [16]:
```
mnist_cnn = CNN(input_channels=1, dropout=DROPOUT)

_, history_mnist_cnn, tl_mnist_cnn, ta_mnist_cnn, time_mnist_cnn = training_loop
    mnist_cnn,
    MNIST_TRAIN_LOADER,
    MNIST_VAL_LOADER,
    MNIST_TEST_LOADER,
    lr=LR,
    epochs=EPOCHS,
    device=DEVICE
)
```

CIFAR-10 MLP

In [17]:
```
cifar_mlp = MLP(input_dim=3*32*32, dropout=DROPOUT)

_, history_cifar_mlp, tl_cifar_mlp, ta_cifar_mlp, time_cifar_mlp = training_loop
    cifar_mlp,
    CIFAR_TRAIN_LOADER,
    CIFAR_VAL_LOADER,
    CIFAR_TEST_LOADER,
    lr=LR,
    epochs=EPOCHS,
    device=DEVICE
)
```

CIFAR-10 CNN

In [18]:
```
cifar_cnn = CNN(input_channels=3, dropout=DROPOUT)

_, history_cifar_cnn, tl_cifar_cnn, ta_cifar_cnn, time_cifar_cnn = training_loop
    cifar_cnn,
    CIFAR_TRAIN_LOADER,
    CIFAR_VAL_LOADER,
    CIFAR_TEST_LOADER,
    lr=LR,
    epochs=EPOCHS,
    device=DEVICE
)
```

A4.2 Results Table

In [19]:
```
results = pd.DataFrame({
    "Dataset": ["MNIST", "MNIST", "CIFAR10", "CIFAR10"],
    "Model": ["MPL", "CNN", "MPL", "CNN"],
    "Test Loss": [
        tl_mnist_mlp,
        tl_mnist_cnn,
        tl_cifar_mlp,
        tl_cifar_cnn
    ],
    "Test Accuracy": [
        ta_mnist_mlp,
        ta_mnist_cnn,
        ta_cifar_mlp,
        ta_cifar_cnn
    ],
    "Training Time Seconds": [
        time_mnist_mlp,
        time_mnist_cnn,
        time_cifar_mlp,
        time_cifar_cnn
    ],
    "Seconds Per Epoch": [     # For Part B
        time_mnist_mlp / EPOCHS,
        time_mnist_cnn / EPOCHS,
        time_cifar_mlp / EPOCHS,
        time_cifar_mlp / EPOCHS
    ]
})

print(results)
```

```
   Dataset Model  Test Loss  Test Accuracy  Training Time Seconds  \
0    MNIST   MPL   0.059394         0.9847             149.112886
1    MNIST   CNN   0.011883         0.9961             212.730535
2  CIFAR10   MPL   1.321178         0.5372             123.665514
3  CIFAR10   CNN   0.521722         0.8239             174.348559

   Seconds Per Epoch
0          14.911289
1          21.273054
2          12.366551
3          12.366551
```

In [20]:
```
print(f"Hardware used: {DEVICE}")
```

```
Hardware used: cuda
```

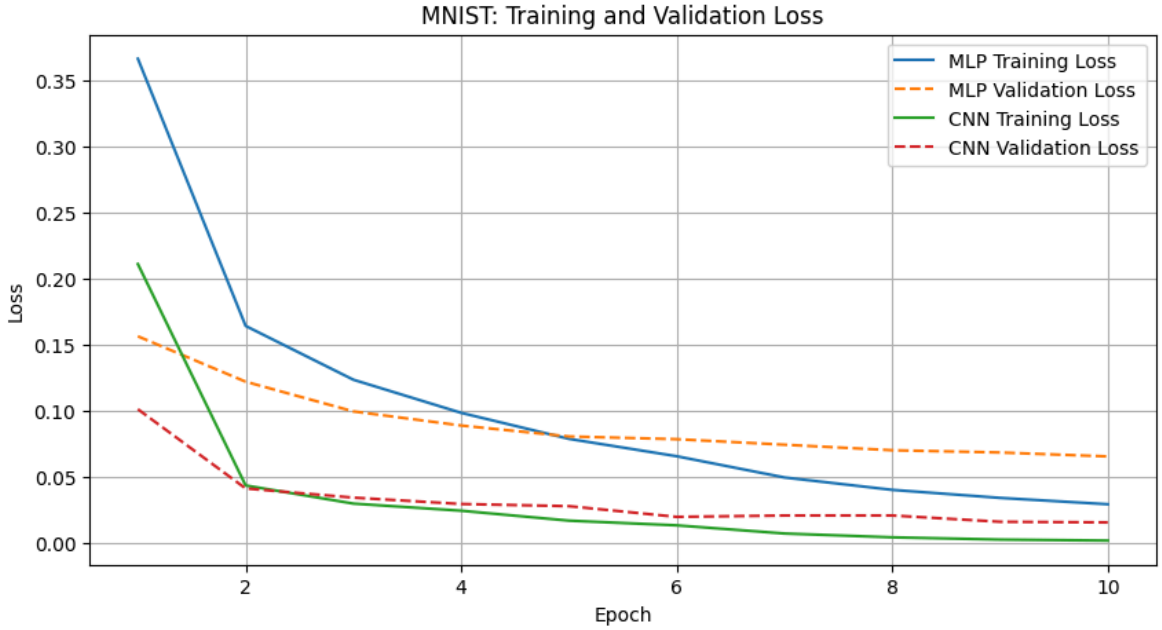
A4.3 PLotting Result Curves

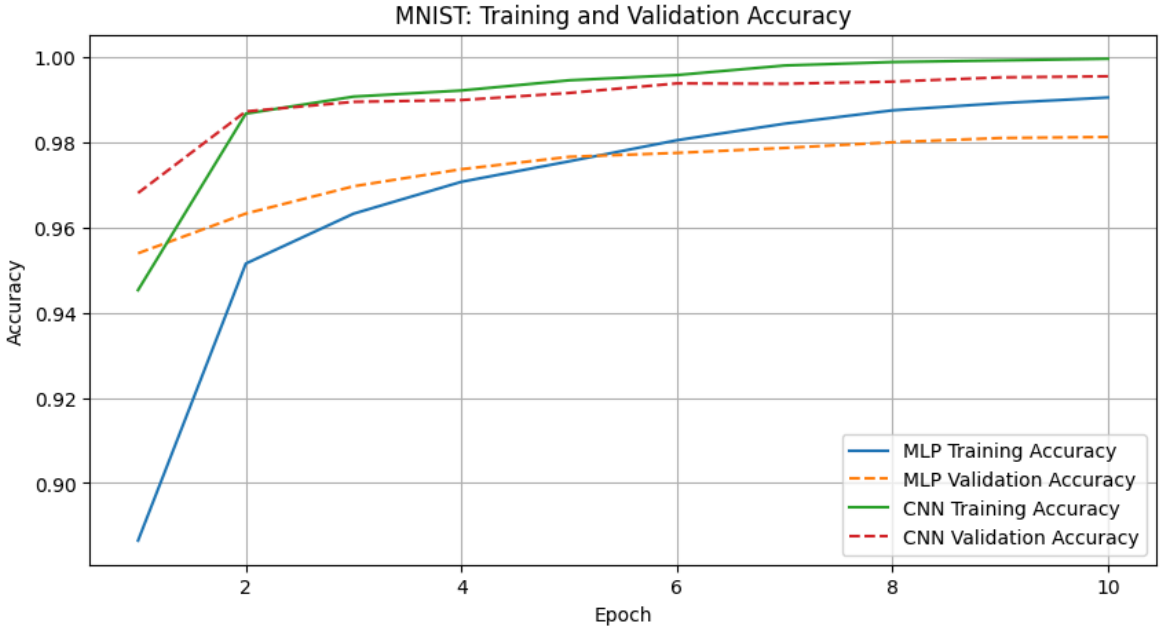
Define plot helper functions

In [21]:
```python
def _plot_curves(dataset_name: str, mlp_history: dict, cnn_history: dict):
    epochs = range(1, len(mlp_history["train_loss"]) + 1)

    # Loss PLot
    plt.figure(figsize=(10, 5))

    plt.plot(epochs, mlp_history["train_loss"], label="MLP Training Loss")
    plt.plot(epochs, mlp_history["val_loss"], label="MLP Validation Loss", lines

    plt.plot(epochs, cnn_history["train_loss"], label="CNN Training Loss")
    plt.plot(epochs, cnn_history["val_loss"], label="CNN Validation Loss", lines

    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.title(f"{dataset_name}: Training and Validation Loss")
    plt.legend()
    plt.grid(True)
    plt.show()

    # Accuracy Loss
    plt.figure(figsize=(10, 5))

    plt.plot(epochs, mlp_history["train_acc"], label="MLP Training Accuracy")
    plt.plot(epochs, mlp_history["val_acc"], label="MLP Validation Accuracy", li

    plt.plot(epochs, cnn_history["train_acc"], label="CNN Training Accuracy")
    plt.plot(epochs, cnn_history["val_acc"], label="CNN Validation Accuracy", li

    plt.xlabel("Epoch")
    plt.ylabel("Accuracy")
    plt.title(f"{dataset_name}: Training and Validation Accuracy")
    plt.legend()
    plt.grid(True)
    plt.show()
```

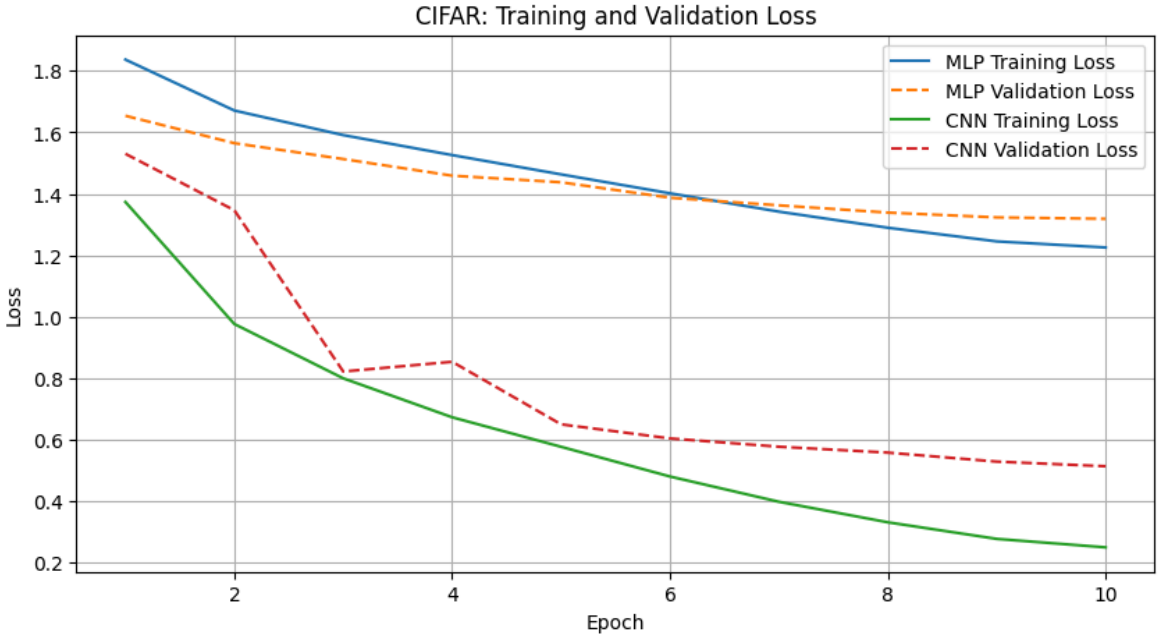
Generate Plots for the Four Experiments

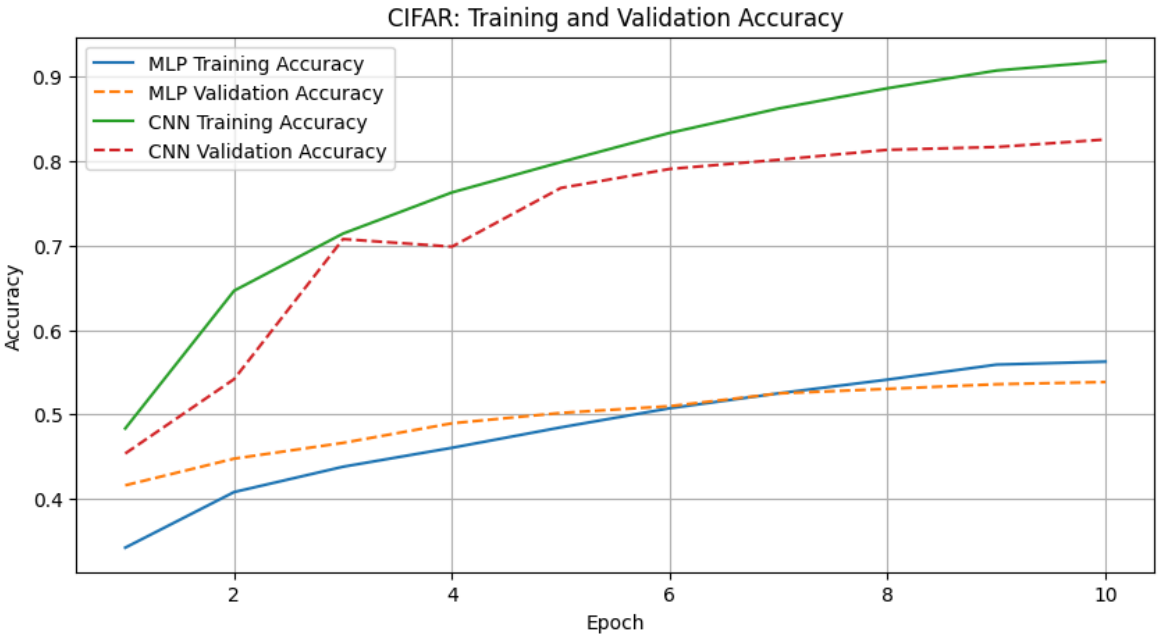
In [22]:
```python
_plot_curves("MNIST", history_mnist_mlp, history_mnist_cnn)
```





In [23]:
```
_plot_curves("CIFAR", history_cifar_mlp, history_cifar_cnn)
```





A4.4 Written Analysis

MNIST

- Looking at the loss curves and the accuracy curves, we can see that the gap between the accuracy of the dataset when using MLP and when using CNN is not that huge. Similar observation can be made from the losses.
- We can see that the dataset performs well under both methods, but slightly better with CNN.
- This can be seen from the results table as the dataset obtains a test accuracy of 98.5% on the CNN model and the test accuracy of 99.6% on the MLP model.
- While CNN provides better accuracy, it sligltly lacks when it comes to performance. The model trains for about 212 seconds while MLP trains for about 149s. For larger datasets, this is bound to scale.
- Either model choice will work well for the MNIST dataset

CIFAR

- Looking at the loss curves and the accuracy curves, we can see that the gap between the accuracy of the dataset when using MLP and when using CNN is quite HUGE. Similar observation can be made from the losses.
- The dataset performs well under CNN, and quite terrible under MLP. This could be due to multi-channel nature of the dataset. Since MLP flattens the image, it does not account for the case of color intensity.
- We can see from the results table that the CIFAR dataset achieves a test accuracy of about 83% under CNN and a test accuracy of about 54% under MLP.
- Similar to the MNIST dataset, the performance of MLP is slighlty better than that of CNN. While MLP trains for about 123 seconds, CNN trains for about 174 seconds.
- The difference is not that huge, meaning it is actually quite better to use CNN for CIFAR-10 dataset.

Complexity Ladder

- The MNIST dataset comprises of simple, grayscale images. This makes the dataset easier to learn.
- The CIFAR-10 dataset contains RGB images, real-world objects with possible messy background, making them diffuclt to learn by simple models.

A4.5 Reflection

The MLP achieved a 99.6% test accuracy with the MNIST dataset and only about 54% test accuracy with the CIFAR-10 dataset. That is a drop of about 45.4%. That is a HUGE drop.

Reasons might be as follows:

- The CIFAR-10 dataset contains RGB images unlike grayscale images in MNIST. The model need not learn only the structure, but also the colors. MLP is not build for that, and hence does not accuratly do the desired.
- The CIFAR-10 dataset contains real-world objects, which may have messy backgrounds. Again, MLP is not desired to account for such cases, and hence model fails to learn features correctly.

PART B: Parameter Budget Challenge

- Use only CIFAR-10
- Design an MLP and CNN with similar parameter counts in the range 475k~525k
- Produce a layer-by-layer parameter table
- Train BOTH models for at least 30 epochs
- Report:
  - Test accuracy
  - Total training time
  - Seconds per epoch
- Hyperparameter search of learning_rate (lr) and num_epochs

- Written Analysis of the Efficiency
- Reflection proposing ONE structural MLP modification

1. Parameter Counting Functions

In [24]:
```
def _count_parameters(model):
    return sum(p.numel() for p in model.parameters() if p.requires_grad)
```

In [25]:
```
def _parameter_table(model: MLP | CNN):
    rows = []

    for name, module in model.named_modules():
        if len(list(module.children())) == 0:
            params = _count_parameters(module)

            if params > 0:
                rows.append({
                    "Layer": name,
                    "Type": module.__class__.__name__,
                    "Trainable Parameters": params
                })

    df = pd.DataFrame(rows)
    total = df["Trainable Parameters"].sum()

    return df, total
```

2. MLP Architecture

For MLP, input size = 3 x 32 x 32 = 3072

AIM: About 475k~500k parameters. How to allocate:

- More parameters for earlier layers
- layer_parameters = layer_in_features * layer_out_features + layer_out_features

Architecture:

- 3 hidden layers and 1 output layer

Implementation:

- h1 = ~400k params
- h2 = ~35k params
- h3 = ~50k params
- out = ~2k params

In [26]:
```
# Define a Budget MLP
class BudgetMLP(nn.Module):
    def __init__(self, input_dim=3072, num_classes: int = 10, dropout: float = 0
        super().__init__()
        self.h1 = nn.Linear(input_dim, 128)
        self.h2 = nn.Linear(128, 256)
        self.h3 = nn.Linear(256, 196)
```

```
        self.out = nn.Linear(196, num_classes)
        self.dropout = nn.Dropout(dropout)      # Define once, reuse

    def forward(self, x):
        x = torch.flatten(x, start_dim=1)

        # First hidden layer
        self.a1 = F.relu(self.h1(x))
        self.a1 = self.dropout(self.a1)

        # Second hidden layer
        self.a2 = F.relu(self.h2(self.a1))
        self.a2 = self.dropout(self.a2)

        # Third hidden layer
        self.a3 = F.relu(self.h3(self.a2))
        self.a3 = self.dropout(self.a3)

        # Output
        return self.out(self.a3)
```

3. CNN Architecture

For CNN, input_channels=3 AIM: About 478k~500k parameters

Architecture: 3 hidden layers, 1 FC layer

In [27]:
```
class BudgetCNN(nn.Module):
    def __init__(self, input_channels=3, num_classes=10, dropout=0.5):
        super().__init__()

        # ── Stage 1: extract low-level features ──────────────────────
        self.stage1 = nn.Sequential(
            ConvBlock(input_channels,  32),
            ConvBlock(32, 32),
            nn.MaxPool2d(2, 2),             # HxW -> H/2 x W/2
        )

        # ── Stage 2: mid-level features ──────────────────────────────
        self.stage2 = nn.Sequential(
            ConvBlock(32, 64),
            ConvBlock(64, 64),
            nn.MaxPool2d(2, 2),             # H/2 x W/2 -> H/4 x W/4
        )

        # ── Stage 3: higher-level features ───────────────────────────
        self.stage3 = nn.Sequential(
            ConvBlock(64, 128),
            ConvBlock(128, 128),
        )

        # ── Classifier head ──────────────────────────────────────────
        self.head = nn.Sequential(
            nn.AdaptiveAvgPool2d((2, 2)),       # 128xH'xW' -> 128x2x2
            nn.Flatten(),                       #  -> 512
            nn.Linear(512, 384),
            nn.ReLU(),
            nn.Dropout(dropout),
```

```
            nn.Linear(384, num_classes),
        )

    def forward(self, x):
        x = self.stage1(x)
        x = self.stage2(x)
        x = self.stage3(x)
        x = self.head(x)
        return x
```

4. Verify Parameter Counts

In [28]:
```
# Define Budget Models
budget_mlp = BudgetMLP().to(DEVICE)
budget_cnn = BudgetCNN().to(DEVICE)

# Get Model parameter counts
mlp_param_count = _count_parameters(budget_mlp)
cnn_param_count = _count_parameters(budget_cnn)

print(f"Budget MLP parameters: {mlp_param_count}")
print(f"Budget CNN parameters: {cnn_param_count}")
```

```
Budget MLP parameters: 478710
Budget CNN parameters: 488298
```

5. Layer-by-Layer Parameter Tables

In [29]:
```
mlp_table, mlp_total = _parameter_table(budget_mlp)
cnn_table, cnn_total = _parameter_table(budget_cnn)
```

MLP Layer-by-Layer Parameter Table

In [30]:
```
print(mlp_table)
```

```
  Layer    Type  Trainable Parameters
0    h1  Linear                393344
1    h2  Linear                 33024
2    h3  Linear                 50372
3   out  Linear                  1970
```

In [31]:
```
print(f"Total MLP parameters: {mlp_total}")
```

```
Total MLP parameters: 478710
```

CNN Layer-by-Layer Parameter Table

In [32]:
```
print(cnn_table)
```

	Layer	Type	Trainable Parameters
0	stage1.0.block.0	Conv2d	864
1	stage1.0.block.1	BatchNorm2d	64
2	stage1.1.block.0	Conv2d	9216
3	stage1.1.block.1	BatchNorm2d	64
4	stage2.0.block.0	Conv2d	18432
5	stage2.0.block.1	BatchNorm2d	128
6	stage2.1.block.0	Conv2d	36864
7	stage2.1.block.1	BatchNorm2d	128
8	stage3.0.block.0	Conv2d	73728
9	stage3.0.block.1	BatchNorm2d	256
10	stage3.1.block.0	Conv2d	147456
11	stage3.1.block.1	BatchNorm2d	256
12	head.2	Linear	196992
13	head.5	Linear	3850

In [33]:
```
print(f"Total CNN parameters: {cnn_total}")
```

```
Total CNN parameters: 488298
```

6. Train BOTH models for 30 epochs

- Reuse functions and loaders defined in PART A

In [34]:
```
BUDGET_EPOCHS = 30
```

Budget MLP

In [35]:
```
final_budget_mlp = BudgetMLP()

_, history_budget_mlp, tl_budget_mlp, ta_budget_mlp, time_budget_mlp = training_
    final_budget_mlp,
    CIFAR_TRAIN_LOADER,
    CIFAR_VAL_LOADER,
    CIFAR_TEST_LOADER,
    lr=LR,
    epochs=BUDGET_EPOCHS,
    device=DEVICE
)
```

Budget CNN

In [36]:
```
final_budget_cnn = BudgetCNN()

_, history_budget_cnn, tl_budget_cnn, ta_budget_cnn, time_budget_cnn = training_
    final_budget_cnn,
    CIFAR_TRAIN_LOADER,
    CIFAR_VAL_LOADER,
    CIFAR_TEST_LOADER,
    lr=LR,
    epochs=BUDGET_EPOCHS,
    device=DEVICE
)
```

```
Early stopping at epoch 15
```

B4.2 Results Table

In [37]:
```
budget_results = pd.DataFrame({
    "Model": ["Budget MPL", "Budget CNN"],
    "Parameters": [
        _count_parameters(final_budget_mlp),
        _count_parameters(final_budget_cnn)
    ],
    "Test Loss": [
        tl_budget_mlp,
        tl_budget_cnn
    ],
    "Test Accuracy": [
        ta_budget_mlp,
        ta_budget_cnn
    ],
    "Training Time Seconds": [
        time_budget_mlp,
        time_budget_cnn
    ],
    "Seconds Per Epoch": [
        time_budget_mlp / BUDGET_EPOCHS,
        time_budget_cnn / BUDGET_EPOCHS
    ]
})

print(budget_results)
```

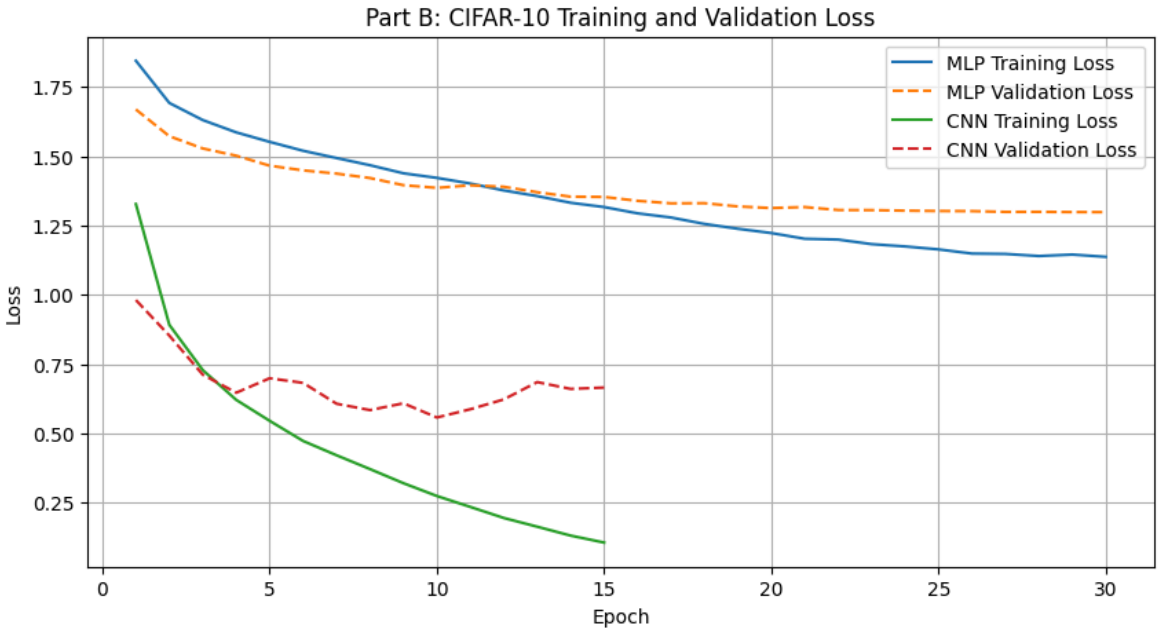
```
       Model  Parameters  Test Loss  Test Accuracy  Training Time Seconds  \
0  Budget MPL      478710   1.296385         0.5429             368.507787
1  Budget CNN      488298   0.695544         0.8181             261.998695

   Seconds Per Epoch
0          12.283593
1           8.733290
```

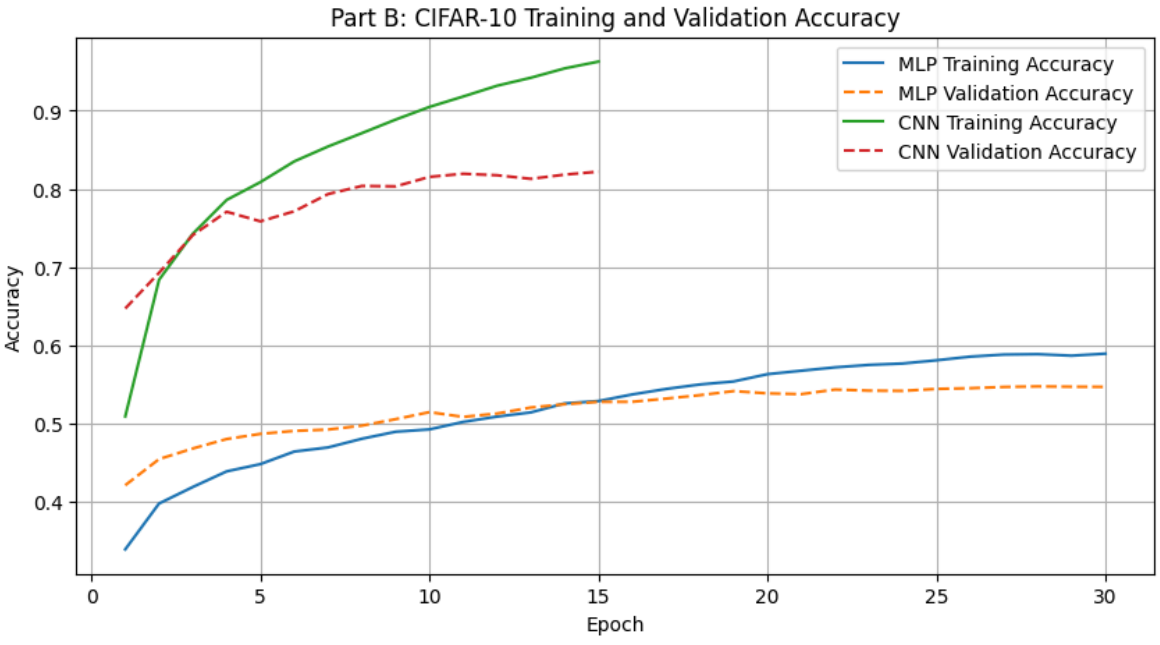
B4.2 Training Curves

In [38]:
```
def _plot_b_curves(mlp_history, cnn_history):
    mlp_epochs = range(1, len(mlp_history["train_loss"]) + 1)
    cnn_epochs = range(1, len(cnn_history["train_loss"]) + 1)

    # Loss PLot
    plt.figure(figsize=(10, 5))

    plt.plot(mlp_epochs, mlp_history["train_loss"], label="MLP Training Loss")
    plt.plot(mlp_epochs, mlp_history["val_loss"], label="MLP Validation Loss", l

    plt.plot(cnn_epochs, cnn_history["train_loss"], label="CNN Training Loss")
    plt.plot(cnn_epochs, cnn_history["val_loss"], label="CNN Validation Loss", l

    plt.xlabel("Epoch")
    plt.ylabel("Loss")
    plt.title(f"Part B: CIFAR-10 Training and Validation Loss")
    plt.legend()
    plt.grid(True)
    plt.show()
```

```
    # Accuracy Loss
    plt.figure(figsize=(10, 5))

    plt.plot(mlp_epochs, mlp_history["train_acc"], label="MLP Training Accuracy"
    plt.plot(mlp_epochs, mlp_history["val_acc"], label="MLP Validation Accuracy"

    plt.plot(cnn_epochs, cnn_history["train_acc"], label="CNN Training Accuracy"
    plt.plot(cnn_epochs, cnn_history["val_acc"], label="CNN Validation Accuracy"

    plt.xlabel("Epoch")
    plt.ylabel("Accuracy")
    plt.title(f"Part B: CIFAR-10 Training and Validation Accuracy")
    plt.legend()
    plt.grid(True)
    plt.show()
```

In [39]:
```
_plot_b_curves(history_budget_mlp, history_budget_cnn)
```





B4.3 Hyperparameter Search

- We will be tuning the following hyperparameters:
  - learning rate
  - dropout
- The model will be trained on 15 epochs first

In [40]:
```
# Tune the Budget MLP model
LRS = [3e-4, 1e-4]
DROPOUTS = [0.5, 0.1]
TUNING_EPOCHS = 15
```

In [41]:
```
def _hyperparameter_search(model_class, model_name: str, train_loader: DataLoade
    results = []

    for lr in LRS:
        for dropout in DROPOUTS:
            model = model_class(dropout=dropout)

            trained_model, history, tl, ta, total_time = training_loop(
                model,
                train_loader,
                val_loader,
                test_loader,
                lr=lr,
                epochs=TUNING_EPOCHS,
                device=device
            )

            best_val_acc = max(history["val_acc"])

            results.append({
                "Model": model_name,
                "lr": lr,
                "dropout": dropout,
                "Best Val Acc": best_val_acc,
                "Test Acc": ta,
                "Seconds Per Epoch": total_time / TUNING_EPOCHS
            })

    return pd.DataFrame(results)
```

MLP Hyperparameter Tuning

In [42]:
```
mlp_search_results = _hyperparameter_search(
    BudgetMLP,
    "Budget MLP",
    CIFAR_TRAIN_LOADER,
    CIFAR_VAL_LOADER,
    CIFAR_TEST_LOADER,
    DEVICE
)

print(mlp_search_results)
```

	Model	lr	dropout	Best Val Acc	Test Acc	Seconds Per Epoch
0	Budget MLP	0.0003	0.5	0.4862	0.4853	12.219523
1	Budget MLP	0.0003	0.1	0.5471	0.5416	12.064606
2	Budget MLP	0.0001	0.5	0.4538	0.4564	11.982207
3	Budget MLP	0.0001	0.1	0.5208	0.5199	12.072613

CNN Hyperparameter Tuning

In [43]:
```
cnn_search_results = _hyperparameter_search(
    BudgetCNN,
    "Budget CNN",
    CIFAR_TRAIN_LOADER,
    CIFAR_VAL_LOADER,
    CIFAR_TEST_LOADER,
    DEVICE
)

print(cnn_search_results)
```

```
Early stopping at epoch 15
```

	Model	lr	dropout	Best Val Acc	Test Acc	Seconds Per Epoch
0	Budget CNN	0.0003	0.5	0.8171	0.8089	17.058451
1	Budget CNN	0.0003	0.1	0.8076	0.8022	17.008306
2	Budget CNN	0.0001	0.5	0.7652	0.7624	17.197274
3	Budget CNN	0.0001	0.1	0.7641	0.7675	17.182251

In [44]:
```
# View all results side by side
tuning_results = pd.concat(
    [mlp_search_results, cnn_search_results],
    ignore_index=True
)

print(tuning_results)
```

	Model	lr	dropout	Best Val Acc	Test Acc	Seconds Per Epoch
0	Budget MLP	0.0003	0.5	0.4862	0.4853	12.219523
1	Budget MLP	0.0003	0.1	0.5471	0.5416	12.064606
2	Budget MLP	0.0001	0.5	0.4538	0.4564	11.982207
3	Budget MLP	0.0001	0.1	0.5208	0.5199	12.072613
4	Budget CNN	0.0003	0.5	0.8171	0.8089	17.058451
5	Budget CNN	0.0003	0.1	0.8076	0.8022	17.008306
6	Budget CNN	0.0001	0.5	0.7652	0.7624	17.197274
7	Budget CNN	0.0001	0.1	0.7641	0.7675	17.182251

Find the Best Configurations

In [45]:
```
best_configs = tuning_results.sort_values(by="Best Val Acc", ascending=False)

print(best_configs)
```

	Model	lr	dropout	Best Val Acc	Test Acc	Seconds Per Epoch
4	Budget CNN	0.0003	0.5	0.8171	0.8089	17.058451
5	Budget CNN	0.0003	0.1	0.8076	0.8022	17.008306
6	Budget CNN	0.0001	0.5	0.7652	0.7624	17.197274
7	Budget CNN	0.0001	0.1	0.7641	0.7675	17.182251
1	Budget MLP	0.0003	0.1	0.5471	0.5416	12.064606
3	Budget MLP	0.0001	0.1	0.5208	0.5199	12.072613
0	Budget MLP	0.0003	0.5	0.4862	0.4853	12.219523
2	Budget MLP	0.0001	0.5	0.4538	0.4564	11.982207

B4.4 Efficiency Analysis

Under parameter constrainsts, the following were discovered:

- Both the CNN and MLP model were designed to fit the same parameter budget. The following results were seen:
  - Budget MLP had 478 710 parameter count while Budget CNN had 488 298 parameter count.
  - Budget MLP achieved a test accuracy of about 54.2% while Budget CNN achieved a test accuracy of about 81.8%

The CNN outperformed the MLP because its parameters are used in convolutional filters that learn local visual features texture, object parts, colour, and edges. These filters are shared across the image, so one learned feature detector can be reused in many spatial positions.

MLP on the other hand, uses many of its parameters in dense layers after flattening. Does not consider the relation of neighbouring pixel values, so the network is forced to learn features from scratch. CNN parameters are therefore highy efficient for images as the architecture naturally models pixel connections, while MLP parameters spend most of the time trying to model them. We can see from the results table, where MLP reports a higher training per epoch of about 12 seconds as compared to the 8 seconds of CNN.

The CNN has a **stronger inductive bias** for images as it assumes nearby pixels are related and that the same visual pattern can appear in different positions.